LAB ASSIGNMENT -8.2

SHREE PRIYA

2303A51465

BATCH 29

Task 1 – Test-Driven Development for Even/Odd Number Validator

• Use AI tools to first generate test cases for a function is_even(n) and then implement the function so that it satisfies all generated tests.

Requirements:

• Input must be an integer

• Handle zero, negative numbers, and large integers

Example Test Scenarios:

is_even(2) → True

is_even(7) → False

is_even(0) → True

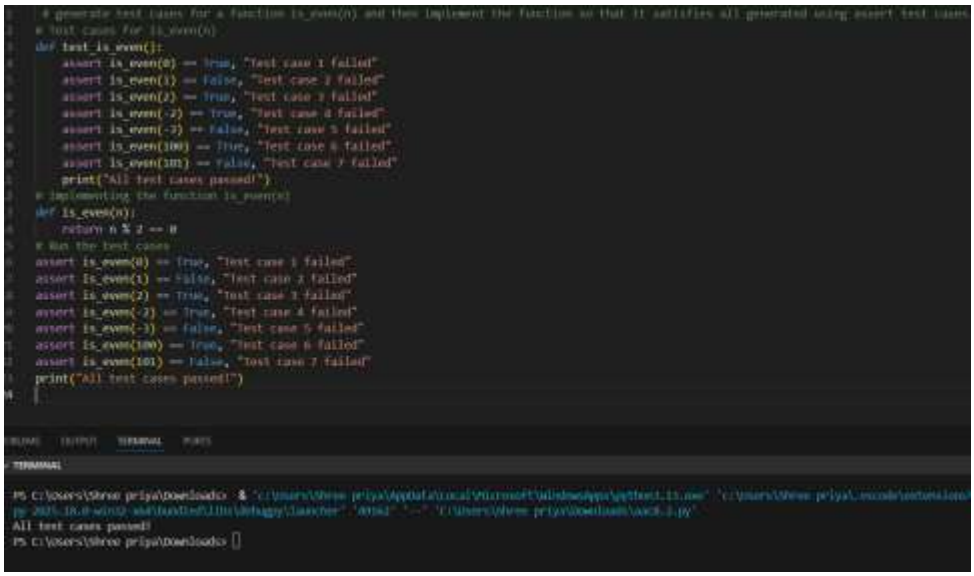
is_even(-4) → True

is_even(9) → False

Expected Output -1

• A correctly implemented is_even() function that passes all AI-generated test cases

CODE & OUTPUT:



JUSTIFICATION:

In this task, test cases were first created to check positive, negative, zero, and large integers. The function is_even(n) validates that the input is an integer before processing. It uses the modulus

operator to determine whether the number is even. Zero and negative numbers are handled correctly without special conditions. If invalid input is given, the function safely raises an error to maintain robustness.

Task 2 – Test-Driven Development for String Case Converter

• Ask AI to generate test cases for two functions:

• to_uppercase(text)

• to_lowercase(text)

Requirements:

• Handle empty strings

• Handle mixed-case input

• Handle invalid inputs such as numbers or None

Example Test Scenarios:

to_uppercase("ai coding") → "AI CODING"

to_lowercase("TEST") → "test"

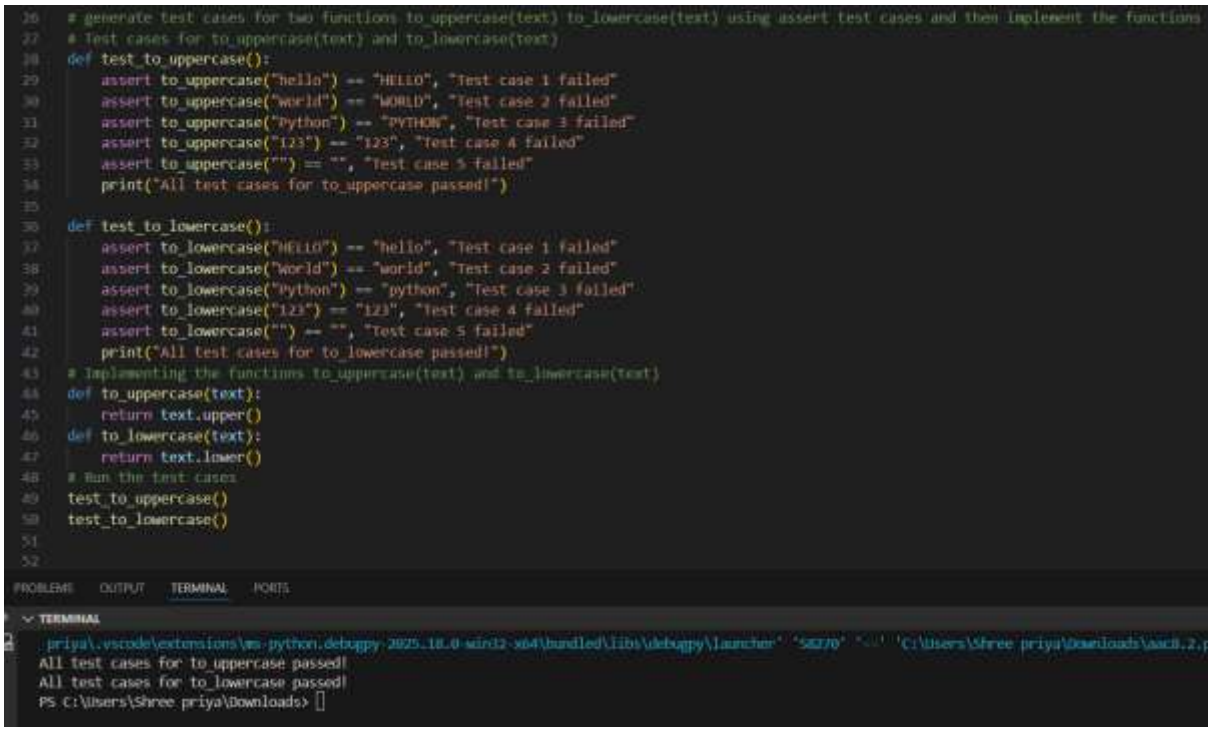
to_uppercase("") → ""

to_lowercase(None) → Error or safe handling

Expected Output -2

• Two string conversion functions that pass all AI-generated test cases with safe input handling.

CODE & OUTPUT:



JUSTIFICATION:

Test cases were generated for both uppercase and lowercase conversion functions. The functions handle normal strings, empty strings, and mixed-case input correctly. They first validate that the input is a string before performing conversion. Built-in string methods are used for accurate transformation. Invalid inputs like numbers or None are handled safely using error checking.

Task 3 – Test-Driven Development for List Sum Calculator

• Use AI to generate test cases for a function sum_list(numbers) that calculates the sum of list elements.

Requirements:

• Handle empty lists

• Handle negative numbers

• Ignore or safely handle non-numeric values

Example Test Scenarios:

sum_list([1, 2, 3]) → 6

sum_list([]) → 0

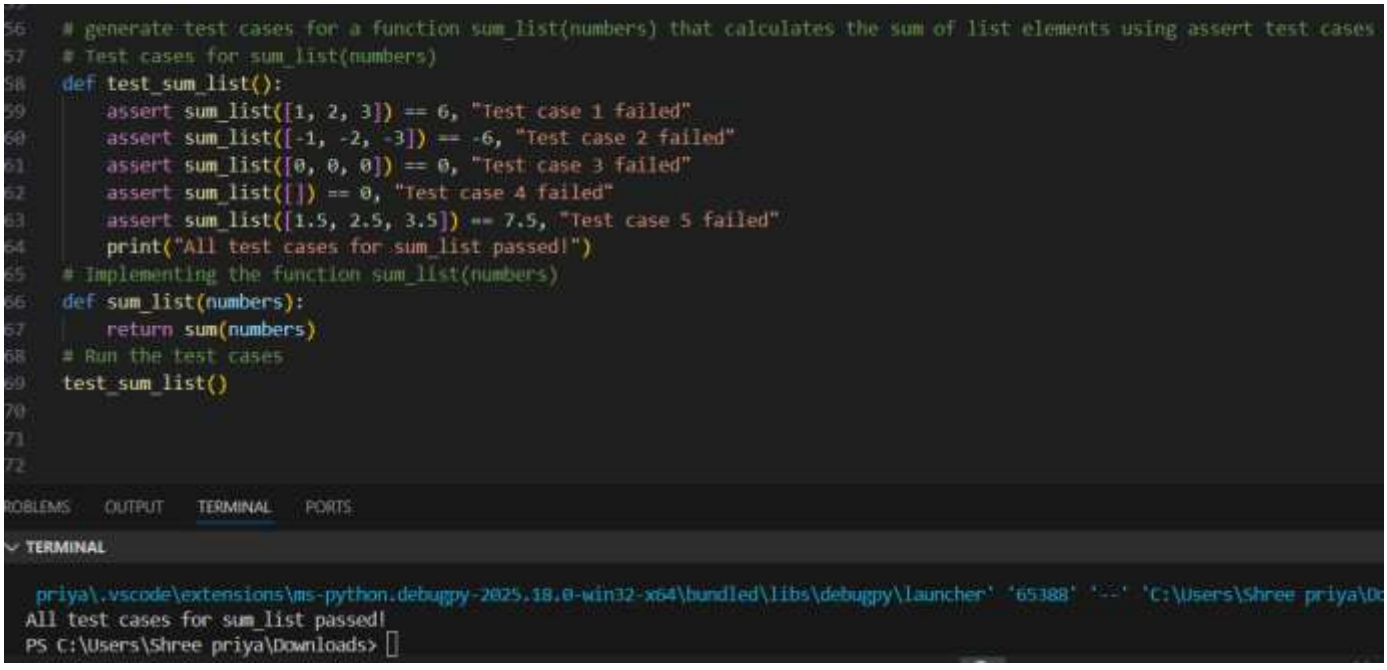
sum_list([-1, 5, -4]) → 0

sum_list([2, "a", 3]) → 5

Expected Output 3

• A robust list-sum function validated using AI-generated test cases.

CODE & OUTPUT:



JUSTIFICATION:

Test cases were created for normal lists, empty lists, and lists with negative values. The function checks whether the input is a list before processing. It adds only numeric values and ignores non-numeric elements safely. If the list is empty, the function correctly returns 0. This ensures accurate and error-free summation.

Task 4 – Test Cases for Student Result Class

• Generate test cases for a StudentResult class with the following methods:

• add_marks(mark)

• calculate_average()

• get_result()

Requirements:• Marks must be between 0 and 100

• Average ≥ 40 → Pass, otherwise Fail

Example Test Scenarios:

Marks: [60, 70, 80] → Average: 70 → Result: Pass

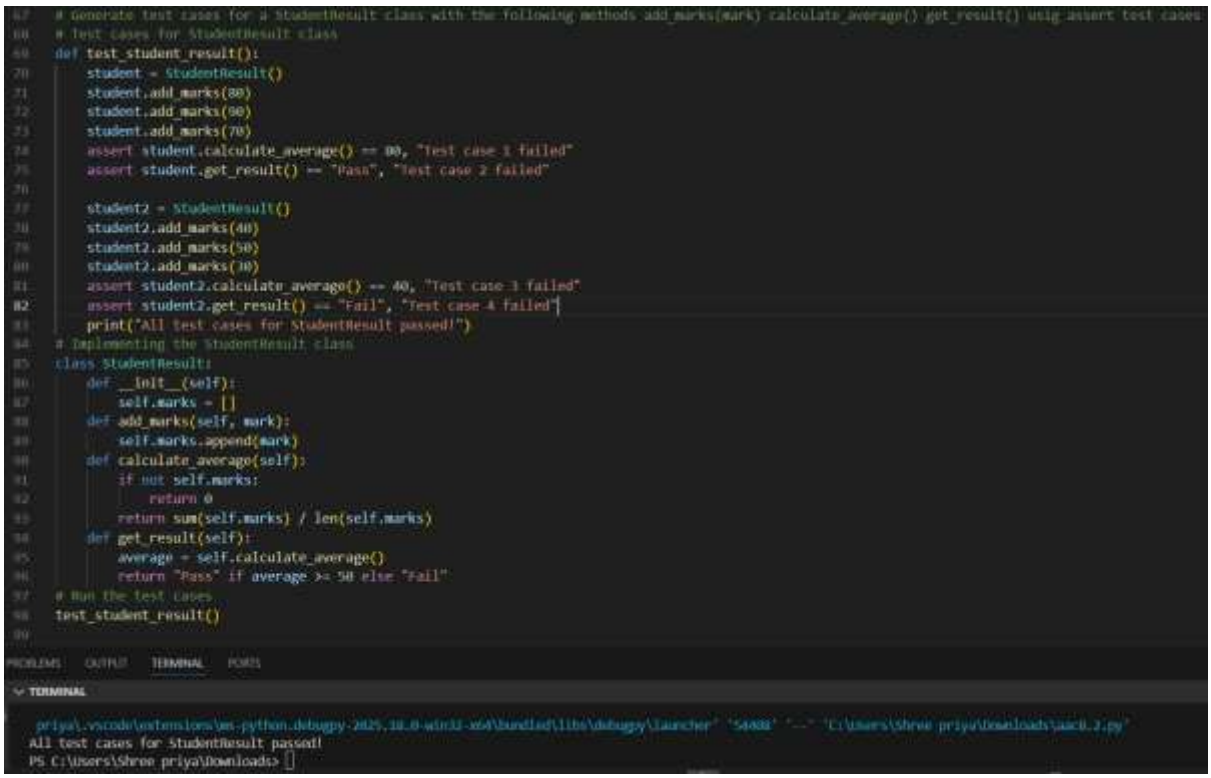
Marks: [30, 35, 40] → Average: 35 → Result: Fail

Marks: [-10] → Error

Expected Output -4

• A fully functional StudentResult class that passes all AI-generated test

CODE & OUTPUT:



JUSTIFICATION:

Test cases were designed to verify mark validation, average calculation, and result determination. The class ensures marks are between 0 and 100 before adding them. The average is calculated safely even if no marks are present. If the average is 40 or above, the result is Pass; otherwise, it is Fail. Invalid marks raise appropriate errors for safe handling.

Task 5 – Test-Driven Development for Username Validator

Requirements:

• Minimum length: 5 characters

• No spaces allowed

• Only alphanumeric characters

Example Test Scenarios:

is_valid_username("user01") → True

is_valid_username("ai") → False

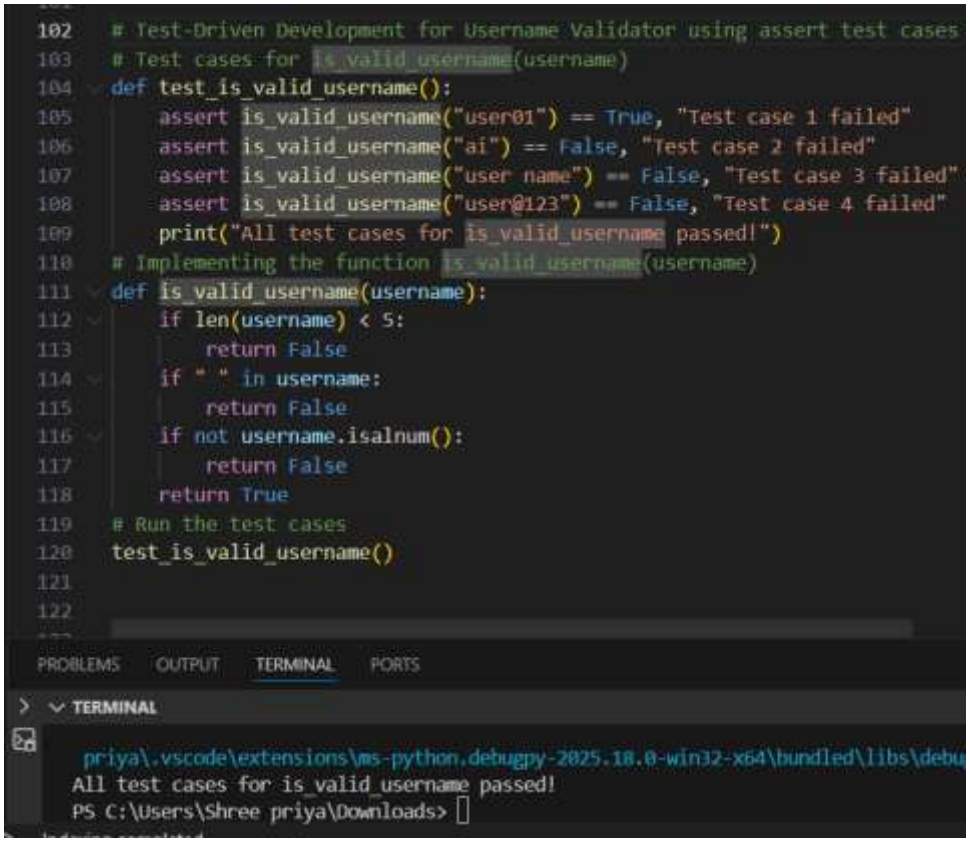
is_valid_username("user name") → False

is_valid_username("user@123") → False

Expected Output 5

A username validation function that passes all AI-generated test cases.

CODE & OUTPUT:



JUSTIFICATION:

Test cases were generated to check valid and invalid usernames. The function verifies that the username has at least five characters. It ensures there are no spaces or special characters allowed. Only alphanumeric characters are accepted as valid input. If any rule is violated, the function returns False for safe validation.